



Banco de Dados Técnicas de Controle de Concorrência

Marcel Oliveira

1



Na Aula Passada...

- Conceitos de transações
- Propriedades desejadas nas transações
 - ACID
- Caracterização de planos de execução
 - baseados em restaurabilidade
 - baseados em seriabilidade
- Protocolos para garantir propriedades

2



Nesta Aula...

- Controle de concorrência para BD
 - Propósitos do controle de concorrência
 - Técnica de bloqueio em duas fases

3



Propósito do Controle de Concorrência

- Forçar o isolamento (através de exclusão mútua) entre transações conflitantes
 - Preservar consistência do BD
 - Resolver conflitos de read-write e write-write
- Exemplo
 - Em um ambiente de execução concorrente, se existe conflito entre T_1 e T_2 sobre um determinado item de dado A, então o controle de concorrência decide se T_1 ou T_2 terá acesso a A, e se a outra transação deverá fazer um *rollback* ou esperar

4



Bloqueio

- Cada item de dado tem um variável de bloqueio que tem um estado
- Nos próximos *slides*
 - Natureza e tipos de bloqueio
 - Protocolos que usam bloqueio para garantir seriability de transações
 - Problemas com o uso de bloqueios

5



Bloqueio Binário

- Pode ter dois valores
 - Bloqueado (locked)
 - Desbloqueado (unlocked)
- Duas operações
 - *lock_item(X)*
 - *unlock_item (X)*

6



Bloqueio Binário

- Bastante simples de implementar
 - Para cada item temos
`<nome do item de dado,
LOCK,
transação de bloqueio>`
 - Manter tabela *hash* com informações sobre itens

7



Bloqueio Binário

- *lock_item(X)*: é uma operação que garante a uma transação

- Permissão de leitura de um item
- Permissão de escrita sobre um item

```
B:
if LOCK(X) = unlocked
then LOCK(X) := locked
else
begin
    wait (until lock(X) = unlocked)
        and the lock manager wakes
        up the transaction);
    goto B
end;
```

8



Bloqueio Binário

- *unlock_item(X)*: é uma operação que remove essas permissões do item de dado

```
LOCK(X) := unlocked
    if any transactions are waiting
    then wake up one of the waiting
        transactions;
```

9



Bloqueio Binário

- Sistema Gerenciador de Bloqueio faz parte do SGBD
- Transações respeitam as seguintes regras
 - *lock_item(X)* deve vir antes de todos os *read_item(X)* ou *write_item(X)*
 - *unlock_item(X)* deve vir depois de que todos *read_item(X)* ou *write_item(X)* sejam completados
 - Não haverá um *lock_item(X)* em uma transação que já tenha o bloqueio sobre X
 - Não haverá um *unlock_item(X)* em uma transação que não tenha o bloqueio sobre X

10



Bloqueio Compartilhado/Exclusivo

- Bloqueio binário é bastante restritivo
 - Apenas uma transação pode acessar um item por vez
 - Leituras poderiam ser feitas concorrentemente

11



Bloqueio Compartilhado/Exclusivo

- Bloqueio de múltiplo-modo
 - Três valores possíveis
 - Bloqueado para leitura (bloqueado compartilhado – *read-locked*)
 - Bloqueado para escrita (bloqueado exclusivo – *write-locked*)
 - Desbloqueado (*unlocked*)
 - Três tipos de operação
 - *read_lock(X)*
 - *write_lock(X)*
 - *unlock(X)*

12



Bloqueio Compartilhado/Exclusivo

- Implementação
 - Para cada bloqueio temos
<nome do item de dado,
LOCK,
número de leituras,
transações de bloqueio>
 - Manter tabela com informações sobre
itens

13



Bloqueio Compartilhado/Exclusivo

- *read_lock(X)*: é uma operação que garante a uma transação
permissão de leitura de um item

```
B:
if LOCK(X) = unlocked then
begin
    LOCK(X) := read-locked;
    no_of_reads(X) := 1;
end
else if LOCK(X) = read-locked then
    no_of_reads(X) := no_of_reads(X) + 1
else
begin
    wait (until LOCK(X)=unlocked
        and the lock manager wakes up the
        transaction);
    go to B
end;
```

14



Bloqueio Compartilhado/Exclusivo

- *write_lock(X)*: é uma operação que garante a uma transação permissão de escrita de um item

```
B:
if LOCK(X) = unlocked then
    LOCK(X) := write-locked;
else
    begin
        wait (until LOCK(X) = unlocked
              and the lock manager wakes
              up the transaction);
        go to B
    end;
```

15



Bloqueio Compartilhado/Exclusivo

- *unlock(X)*: é uma operação que remove essas permissões do item de dado

```
if LOCK(X) = write-locked then
begin
    LOCK(X) := unlocked;
    wake up one of the transactions, if any
end
else if LOCK(X) = read-locked then
begin
    no_of_reads(X) := no_of_reads(X) - 1
    if no_of_reads(X) = 0 then
begin
    LOCK(X) := unlocked;
    wake up one of the transactions, if any
end
end
end;
```

16



Bloqueio Compartilhado/Exclusivo

- Transações respeitam as seguintes regras
 - *read_lock(X)* ou *write_lock(X)* deve vir antes de todos os *read_item(X)*
 - *write_lock(X)* deve vir antes de todos os *write_item(X)*
 - *unlock_item(X)* deve vir depois de que todos *read_item(X)* ou *write_item(X)* sejam completados
 - Não haverá um *read_lock(X)* em uma transação que já tenha o bloqueio de leitura sobre *X*
 - Não haverá um *write_lock(X)* em uma transação que já tenha o bloqueio de escrita ou de leitura sobre *X*
 - Não haverá um *unlock(X)* em uma transação que não tenha o bloqueio de leitura ou de escrita sobre *X*

17



Conversão de Bloqueios

- Relaxa as condições
 - Não haverá um *read_lock(X)* em uma transação que já tenha o bloqueio de leitura sobre *X*
 - Não haverá um *write_lock(X)* em uma transação que já tenha o bloqueio de escrita ou de leitura sobre *X*
- Promoção e rebaixamento de bloqueios

18



Conversão de Bloqueios

- Promoção de bloqueios

```
if Ti has a read-lock (X)
    and Tj has no read-lock(X) (i ≠ j)
then
    LOCK(X) := write-lock;
else
    force Ti to wait until Tj unlocks X
```

19



Conversão de Bloqueios

- Rebaixamento de bloqueios

```
if Ti has a write-lock(X)
    LOCK(X) := read-lock;
    no_of_reads(X) := 1
```

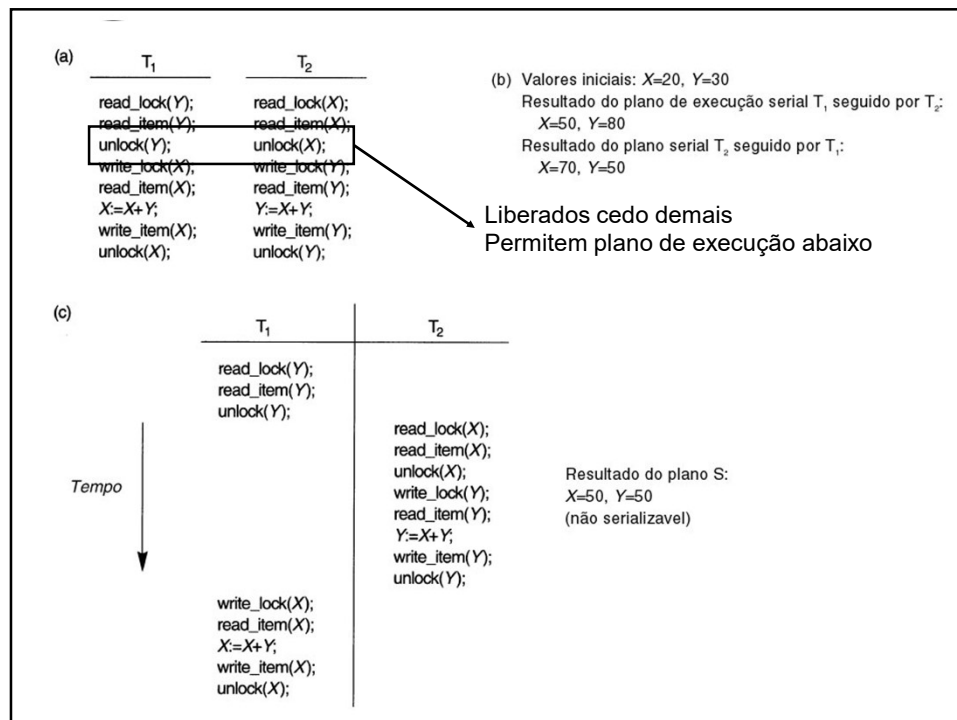
20



Garantindo Serialização

- O uso de bloqueios binários não garante serialização de planos de execução
- Exemplo...

21



22



Garantindo Serialização

- Precisamos seguir um protocolo adicional para garantir a serialização
- Este protocolo está relacionado com o posicionamento das operações de bloqueio e desbloqueio em todas as transações

23



Bloqueio em Duas Fases

- Protocolo de bloqueio em duas fases
 - Todas as operações de *read_lock* e *write_lock* precedem a primeira operação de *unlock*
- Transação com duas fases
 - Fase de expansão: aquisição de bloqueios
 - Fase de encolhimento: liberação de bloqueios

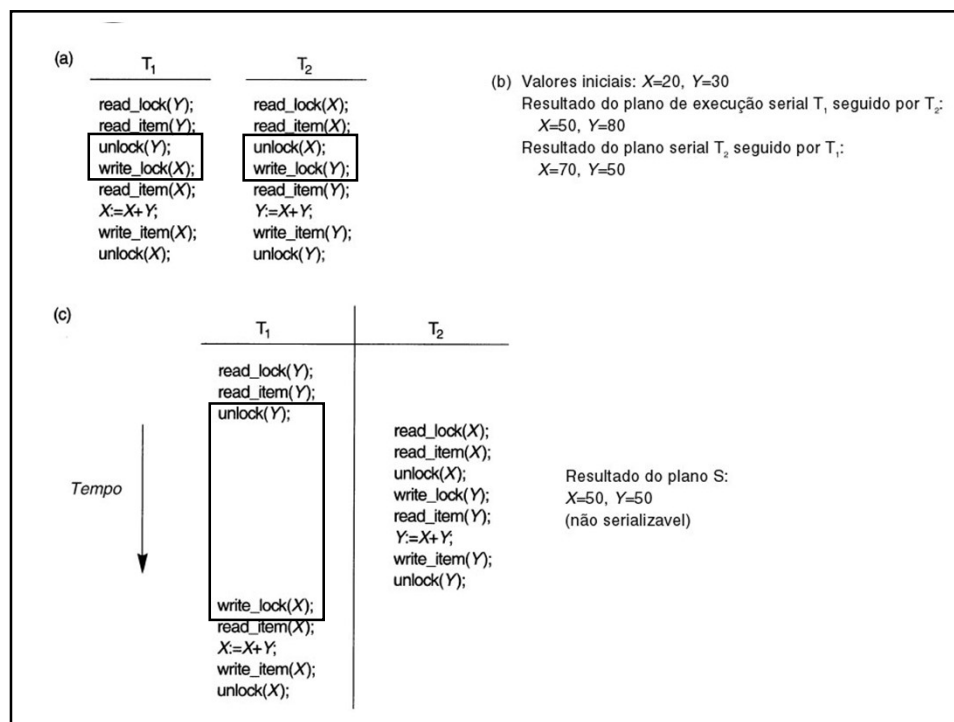
24



Bloqueio em Duas Fases

- Com conversão de bloqueios
 - Fase de expansão
 - Aquisição de bloqueios
 - Promoção de bloqueios
 - Fase de encolhimento
 - Liberação de bloqueios
 - Rebaixamento de bloqueios
- Voltemos ao nosso exemplo...

25

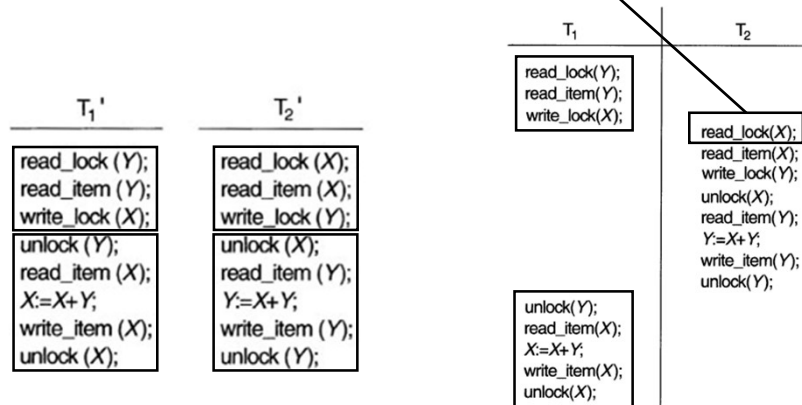


26



Bloqueio em Duas Fases

Forçado a esperar o unlock(X) de T1



27



Bloqueio em Duas Fases

- Podemos provar que um plano de execução que segue o protocolo de bloqueio em duas fases é serializável
 - Não precisamos de teste de seriabilidade
- Preço a pagar
 - Limitamos a quantidade de concorrência que poderia surgir em um plano de execução

28



Variações do Bloqueio em Duas Fases

- Vimos o 2PL básico (*two-phase locking*)
- 2PL Conservador
 - Uma transação deve bloquear todos os itens antes de começar a execução
 - Livre de *deadlocks*
 - Porém, difícil de usar na prática visto que a obtenção de todos os bloqueios diante mão geralmente não é possível

29



Variações do Bloqueio em Duas Fases

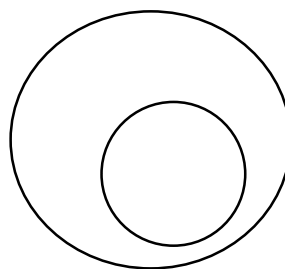
- 2PL Estrito
 - Variação mais popular
 - Não libera nenhum de seus bloqueios de escrita antes de sua efetivação ou cancelamento
 - Não é livre de *deadlocks*
- 2PL Rigoroso
 - Não libera nenhum de seus bloqueios antes de sua efetivação ou cancelamento

30



Bloqueio em Duas Fases

- Protocolo de bloqueio em duas fases garante a serialização do plano
- Porém não permite todos os planos serializáveis



Planos serializáveis

Planos permitidos pelo protocolo

31



Bloqueio em Duas Fases

- Podem gerar dois problemas
 - *Deadlock*
 - *Starvation*

32



Deadlock ???



33



Não vai acontecer de novo???

)

Busca

FOTO: Chuva, falta de energia e caos em diversas regiões de Natal



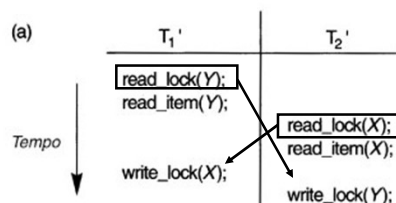
34



Lidando com *Deadlocks*

- *Deadlock*

- Todas as transações de um conjunto de duas (ou mais) transações estão esperando por algum item que está bloqueado por outra transação



35



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de prevenção de *deadlocks*
 - Bloqueio de duas fases conservador (2PL conservador)
 - Bloqueia todos os itens antes de começar a execução
 - Limita demais a concorrência
 - Ordenar itens e bloquear seguindo esta ordem
 - Também não é prático para BDs



36



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de prevenção de *deadlocks*
 - Técnicas com conceitos de marca de tempo (*timestamp*) para cada transação
 - Marcas são baseadas na ordem de início das transações
 - $TS(T_i) < TS(T_j)$ (T_i é mais velha que T_j)
 - Exemplo (Técnica Esperar-Morrer)
 - Se T_i tenta bloquear X, mas X já está bloqueado por T_j
 - » Se $TS(T_i) < TS(T_j)$ então T_i pode esperar
 - » Senão aborta T_i e reinicia mais tarde com a mesma marca



37



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de prevenção de *deadlocks*
 - Protocolo sem espera
 - Se uma transação não puder obter um bloqueio, ela aborta e é reiniciada depois de um certo tempo
 - Protocolo de espera cuidadosa
 - Se uma transação não puder obter um bloqueio por causa de uma outra transação T, ela aborta apenas se T também estiver esperando por algum outro item bloqueado



38



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de detecção de *deadlocks*
 - Protocolos vistos geralmente não são usados na prática
 - Sobrecarga de processamento
 - Detecção é uma abordagem mais prática
 - Verifica se existe um deadlock



39



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de detecção de *deadlocks*
 - Se existirem várias transações, acessando vários itens, concorrentemente e de maneira conflitante
 - Melhor tomar a abordagem da prevenção

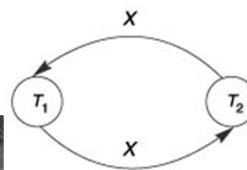


40



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de detecção de *deadlocks*
 - Uso de grafos de espera
 - Identificação de *deadlocks*
 - Escolha de vítimas



41



Lidando com *Deadlocks*

- Podemos prevenir deadlocks usando algoritmos de *timeout*
 - Baixa sobrecarga e simples
 - *Timeout* para o período de espera por um bloqueio
 - Aborta a transação após *timeout*

42



Lidando com *Starvation*

- Uma transação não pode continuar por um período indefinido de tempo, enquanto outras transações estão executando normalmente
 - Soluções
 - Espera imparcial
 - FIFO
 - Transações com prioridades
 - Aumento do tempo de espera aumenta a prioridade da transação
 - Esperar-morrer evita a inanição

43



Outros Controles de Concorrência

- Baseados em ordenação por marca de tempo (*timestamp*)
 - Ordena as transações usando marcas de tempo
- Multiversão
 - Mantém diversas versões de cada item
- Otimista
 - Assumem que há pouca interferência entre as transações
 - Trabalha sobre cópias dos itens
 - Três fases
 - Leitura
 - Validação
 - Escrita

44



Resumo

- Propósito do controle de concorrência
- Bloqueios
 - Bloqueio
 - Bloqueio binário
 - Bloqueio compartilhado/exclusivo
 - Conversão de bloqueios
- Bloqueios em duas fases
 - Fase de expansão
 - Fase de encolhimento

45



Resumo

- Variações do bloqueios em duas fases
 - 2PL Conservador
 - 2PL Estrito
 - 2PL Rigoroso
- Lidando com *deadlock* e *starvation*

46



Leitura

- Capítulos 22

47



48